# Контест 7 · Тесты и логи

Неделя 14 · темы недель 13–14: отчёты тестов, покрытие и мутанты, логи Nginx и JSON, метрики и SLO, алерты, инциденты. Сдача устная у экрана: показываете код, запускаете на примерах, объясняете идею.

> Монеты: обычная задача — **2**, со звёздочкой (★, например B4*) — **4**, каждая часть босса — **6**. В этом контесте: 15 обычных, 3 со звёздочкой, 3 частей босса — всего **60** монет. Задача засчитывается, только если вы можете объяснить свой код.

## A. Тесты

**A1.** `junit_summary(xml)` — разберите отчёт `pytest --junitxml` (модуль `xml.etree.ElementTree`). У каждого `<testcase>` внутри может быть `<failure>`, `<error>` или `<skipped>`; иначе тест прошёл. Ответ — счётчики и суммарное `time`, округлённое до 3 знаков.

```
Ввод:  <testsuites><testsuite name="pytest">
         <testcase name="a" time="0.010"/>
         <testcase name="b" time="0.200"><failure message="assert 1 == 2"/></testcase>
         <testcase name="c" time="0.005"><skipped/></testcase>
         <testcase name="d" time="1.300"><error message="fixture"/></testcase>
         <testcase name="e" time="0.020"/>
       </testsuite></testsuites>
Вывод: {"passed": 2, "failed": 1, "errors": 1, "skipped": 1, "time": 1.535}
```

**A2.** `flaky(runs)` — результаты нескольких запусков CI: словари `тест → прошёл ли`. Flaky — тест, который хотя бы раз прошёл и хотя бы раз упал. По имени.

```
Ввод:  flaky([{"a": True, "b": True, "c": False}, {"a": True, "b": False, "c": False}, {"a":
True, "b": True}])
Вывод: ["b"]
```

**A3.** `missing_ranges(covered, total, skip=set())` — колонка *Missing* отчёта покрытия: строки `1..total`, которых нет ни в `covered`, ни в `skip` (пустые и комментарии), подряд идущие — диапазоном.

```
Ввод:  missing_ranges({1, 2, 6, 7, 10}, 12, skip={4})
Вывод: "3, 5, 8-9, 11-12"
```

**A4*.** `min_tests(kills)` — для каждого мутанта известно, какие тесты его убивают. Найдите наименьший набор тестов, который убивает всех убиваемых мутантов (у кого пустой набор — пропускаем). Из нескольких наборов одного размера — первый в лексикографическом порядке; ответ по имени. Тестов не больше 15.

```
Ввод:  min_tests({"m1": {"t1", "t3"}, "m2": {"t2"}, "m3": {"t3"}, "m4": set(), "m5": {"t1",
"t2"}})
Вывод: ["t2", "t3"]
```

## B. Логи

Строка лога Nginx в формате `combined`:

```
10.0.0.5 - - [09/Feb/2027:10:00:01 +0300] "GET /api/top HTTP/1.1" 200 512 "-" "curl/8.5"
```

**B1.** `parse_access(line)` — словарь `ip, time, method, path, status, bytes, ua`; `status` и `bytes` — числа, `bytes` равное `-` — это `0`. Строка не в формате — `None`.

```
Ввод:  parse_access(строка выше)
Вывод: {"ip": "10.0.0.5", "time": "09/Feb/2027:10:00:01 +0300", "method": "GET",
        "path": "/api/top", "status": 200, "bytes": 512, "ua": "curl/8.5"}
```

**B2.** `top_ips(lines, n)` — `n` адресов с наибольшим числом запросов: `(ip, сколько)`, при равенстве — по ip как строке. Битые строки пропускаем.

```
# L — строка выше, L7 и L2 — она же с ip 10.0.0.7 и 10.0.0.2
Ввод:  top_ips([L, L7, L, "мусор", L2, L7], 2)
Вывод: [("10.0.0.5", 2), ("10.0.0.7", 2)]
```

**B3.** `trace(records, request_id)` — записи разных сервисов с полем `rid`. Соберите путь запроса: записи с этим `rid` по `ts`, при равенстве — в исходном порядке; строки вида `"сервис: msg"`.

```
Ввод:  trace([{"ts": 3, "service": "bot", "rid": "r1", "msg": "reply sent"},
              {"ts": 1, "service": "nginx", "rid": "r1", "msg": "GET /ws"},
              {"ts": 2, "service": "api", "rid": "r2", "msg": "other"},
              {"ts": 2, "service": "api", "rid": "r1", "msg": "join R1"}], "r1")
Вывод: ["nginx: GET /ws", "api: join R1", "bot: reply sent"]
```

## C. Метрики

**C1.** `counter_rate(samples)` — замеры счётчика запросов `(ts, value)`. Счётчик только растёт, но при перезапуске сервиса начинается с нуля: если значение уменьшилось, прирост за этот шаг равен новому значению. Ответ — прирост в секунду за всё время; меньше двух замеров — `None`.

```
Ввод:  counter_rate([(0, 100), (10, 150), (20, 20), (30, 40)])
Вывод: 3.0                    # 50 + 20 + 20 за 30 секунд
```

**C2.** `uptime(checks, interval)` — результаты проверки `/health` каждые `interval` секунд. Ответ — `(доступность в %, округлить до 2 знаков; самый долгий простой в секундах)`. Пусто — `ValueError`.

```
Ввод:  uptime([True, True, False, False, False, True, False, True], 30)
Вывод: (50.0, 90)
```

**C3.** `error_budget(total, failed, slo)` — SLO `0.995` значит: можно ошибиться в 0,5 % запросов. Ответ — `(можно ошибок, осталось, какая доля бюджета потрачена — 3 знака)`; «можно» — целое число, вниз. Остаток может быть отрицательным.

```
Ввод:  error_budget(10000, 20, 0.995), error_budget(200000, 1500, 0.999)
Вывод: (50, 30, 0.4), (200, -1300, 7.5)
```

**C4*.** `burn_alert(minutes, slo, factor=14.4, long=60, short=5)` — по минутам `(запросов, ошибок)`. Скорость сгорания бюджета за окно — `(ошибок / запросов) / (1 − slo)`; окно — последние `long` (или `short`) минут, включая текущую, а в начале — сколько есть. Найдите первую минуту, где скорость **больше** `factor` в обоих окнах, или `None`. Пустое окно — скорость 0.

```
Ввод:  burn_alert([(100, 0)] * 30 + [(100, 30)] * 10, 0.999)
Вывод: 31                     # в минуту 30 короткое окно уже горит, длинное ещё нет
```

## D. Инциденты

**D1.** `downtime(intervals)` — простои `(начало, конец)` из разных алертов могут пересекаться и касаться. Ответ — `(слитые интервалы по возрастанию, общий простой)`.

```
Ввод:  downtime([(10, 20), (15, 30), (40, 45), (30, 35)])
Вывод: ([(10, 35), (40, 45)], 30)
```

**D2.** `dedupe_alerts(alerts, window)` — алерты `(ts, имя)`. Повтор того же имени раньше, чем через `window` секунд после **последнего оставленного**, выбрасываем. Ответ — оставленные по времени.

```
Ввод:  dedupe_alerts([(0, "5xx"), (30, "5xx"), (65, "5xx"), (10, "slow"), (70, "5xx"), (130,
"5xx")], 60)
Вывод: [(0, "5xx"), (10, "slow"), (65, "5xx"), (130, "5xx")]
```

**D3.** `timeline(events)` — хронология для постмортема: события `(ts, текст)` по времени (равные — в исходном порядке), время — от первого события: `T+мм:сс`, от часа — `T+ч:мм:сс`.

```
Ввод:  timeline([(1000, "починили"), (700, "алерт"), (640, "деплой v0.8"), (4300,
"постмортем")])
Вывод: ["T+00:00 деплой v0.8", "T+01:00 алерт", "T+06:00 починили", "T+1:01:00 постмортем"]
```

## E. Проект и алгоритмы

**E1.** На staging логи сервера — JSON одной строкой (лаба 14), есть `/health` и `/metrics` с RED-метриками. Покажите, как найти все ошибки одной комнаты за последние 10 минут одной командой.

**E2.** Алерты в отдельный Telegram-чат команды: правило с `for`, без повторов чаще раза в час, сообщение «решено». Покажите алерт на учении «сайт упал» (остановите контейнер на staging).

**E3.** Постмортем учения в `docs/incidents/`: хронология, причина, что сработало, что нет, три действия с ответственными. Логи разбирал агент — отметьте, что он нашёл верно и что пропустил.

**E4 · босс.** Мини-язык запросов к логам. Записи — словари; уровни сравниваются по порядку `DEBUG < INFO < WARNING < ERROR < CRITICAL`. За каждую часть свой плюс.

а) `parse_query(q)` — условия через пробел: `поле оператор значение`, операторы `= != > >= < <=`. Значение — целое, дробное или строка. Непонятное условие или неизвестный уровень в `level` — `ValueError`.

```
Ввод:  parse_query("level>=WARNING room=R1 ms>100")
Вывод: [("level", ">=", "WARNING"), ("room", "=", "R1"), ("ms", ">", 100)]
```

б) `run_query(records, q)` — записи, где верны все условия. Нет поля — условие ложно; `> >= < <=` — только для чисел (и для `level`), иначе ложно.

```
Ввод:  R = [{"level": "INFO", "path": "/ws", "ms": 20, "room": "R1"},
            {"level": "ERROR", "path": "/ws", "ms": 350, "room": "R1"},
            {"level": "WARNING", "path": "/api", "ms": 120, "room": "R2"},
            {"level": "ERROR", "path": "/api", "ms": 900},
            {"level": "DEBUG", "path": "/ws", "ms": 5, "room": "R1"}]
       run_query(R, "level>=WARNING room=R1 ms>100")
Вывод: [{"level": "ERROR", "path": "/ws", "ms": 350, "room": "R1"}]
```

в) `stats(records, q)` — `"<условия> | count by F"`, `"| avg X by F"` (округлить до одного знака), `"| max X by F"`. Записи без `F` пропускаем. Ключи по возрастанию. Другое — `ValueError`.

```
Ввод:  stats(R, "level>=INFO | count by path"), stats(R, "| avg ms by path")
Вывод: {"/api": 2, "/ws": 2}, {"/api": 510.0, "/ws": 125.0}
```

**E5*.** `merge_logs(files)` — логи нескольких сервисов, в каждом строки JSON с `ts`, но бывают строки без `ts` (трейсбек) — они относятся к предыдущей записи своего файла и идут вместе с ней; строки до первой записи выбрасываем. Слейте всё по `ts`; при равенстве — по номеру файла, затем по порядку в файле.

```
Ввод:  a = ['{"ts": 1, "msg": "api start"}', '{"ts": 5, "msg": "api error"}',
            'Traceback (most recent call last):', '  ValueError: boom']
       b = ['{"ts": 2, "msg": "bot start"}', '{"ts": 5, "msg": "bot retry"}']
       merge_logs([a, b])
Вывод: [a[0], b[0], a[1], a[2], a[3], b[1]]
```